

OnWork 그룹웨어

Architecture Decision Records

3차 산출물

강희진 · 김성민 · 이서영 · 이종하

포함된 ADR

ID	제목
ADR-SYS-001	아키텍처 패턴: 계층화(Layered) 선택
ADR-SYS-002	인증 방식: JWT(Access + Refresh Token) 선택
ADR-HR-001	인사 변경 승인 전 실제 데이터 미반영
ADR-HR-002	인사 변경 요청에 반려(REJECTED) 구조 유지
ADR-HR-003	퇴사 처리 시 계정 삭제 대신 비활성화
ADR-ATT-001	결근 감지: 자정 배치 자동 처리
ADR-ATT-002	미승인 시간외 근로: 이상 기록 후 사후 확정 구조
ADR-LVE-001	휴가 승인 흐름: 단일 승인 + 부재 시 대리자 자동 위임
ADR-LVE-002	휴가 승인 처리: 반려 없이 승인/보류만 구현
ADR-LVE-003	보상휴가 다중 부여: 배열 입력 + 부분 성공(Partial Success) 허용
ADR-APP-001	결재함 통합 뷰
ADR-APP-002	온보딩 유형 2종 제한 결정
ADR-APP-003	시스템 자동 트리거 방식 채택
ADR-APP-004	튜토리얼 정의의 코드 기반 설계

ADR-SYS-001 아키텍처 패턴: 계층화(Layered) 선택

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	전체 시스템 (Auth, HR, 근태, 휴가, 일정, 알림)

배경 (Context)

온워크는 상시 근로자 20명 내외의 한국 중소기업을 위한 그룹웨어로, 근태·휴가·인사·일정 관리를 하나의 시스템에서 제공한다. 백엔드 팀원 4명이 병렬 개발하며, 초기 버전에서는 복잡한 분산 처리보다 관심사의 분리와 팀 분업 최적화가 우선 과제다.

선택지로 검토한 아키텍처 패턴은 다음과 같다.

패턴	장점	단점
① 계층화(Layered) ★ 채택	변경의 격리 · 팀 분업 최적화 · 낮은 진입 장벽	모듈 간 물리적 분리 없음 → 패키지 경계로 결합도 통제 필요
② MSA	모듈별 독립 배포 · 특정 서비스 스케일아웃 가능	4명 팀에 Cognitive Load 과도 · 네트워크 레이턴시 · 운영 복잡도 급증
③ 모놀리식(단순)	개발 속도 최단 · 배포 단순	단순 조회 병목을 캐시로만 완화해야 함 · 향후 분리 어려움

결정 (Decision)

Presentation → Business → Data Access의 3계층 Layered Architecture를 채택한다.

각 계층은 항상 상위→하위 단방향 의존성을 유지하며, Data Access 계층은 인터페이스를 통해 DB 기술 의존을 격리한다. 도메인 모듈(Auth, HR, 근태, 휴가, 일정, 알림)은 계층 내부에서 패키지 경계로 분리하고, 모듈 간 직접 참조 대신 서비스 인터페이스를 통해 통신한다.

이유 (Rationale)

- 수업에서 강조한 '변경의 격리 + 팀 분업 최적화 + 낮은 진입 장벽' 3가지 장점이 4인 소규모 팀에 부합
- CRUD 중심의 비즈니스 로직이 주류인 그룹웨어 특성상 계층화가 충분한 설계 유연성을 제공
- MSA는 현 팀 규모에서 Cognitive Load가 과도하고, 운영 인프라 비용(k8s, 서비스 메시 등)이 20인 고객사 니즈와 불균형
- 단순 모놀리식은 향후 특정 모듈(예: 알림) 분리 요구 시 리팩터링 비용이 큼

대가 (Consequences)

감수하는 트레이드오프

- 모듈 간 물리적 분리가 없으므로, 패키지 접근 제어 규칙을 코드 리뷰에서 수동으로 검증해야 함
- 향후 트래픽 급증 시 특정 모듈만 스케일아웃하기 어려움 → 수평 확장은 전체 서버 단위로 이뤄짐

완화 방안

- Redis 캐시 도입으로 단순 조회 병목을 선제 완화
- 패키지 경계 위반 감지를 위한 정적 분석 룰(ArchUnit 등) 도입을 v2 로드맵에 포함

설계 원리 적용 명세

SRP (단일 책임 원칙)

- ApprovalService — 결재함 조회 및 라우팅만 담당
 - LeaveService — 휴가 비즈니스 로직만 담당
 - AttendanceService — 근태 비즈니스 로직만 담당
 - HrService — 인사 비즈니스 로직만 담당
- 이유: ApprovalService가 LeaveService의 잔여일수 차감 로직까지 알면 휴가 정책 변경 시 ApprovalService도 수정해야 한다. 위임 구조를 유지하면 변경의 파급 범위가 각 모듈 내부로 한정된다.

고응집 / 저결합

- 도메인 모듈은 계층 내부에서 패키지 경계로 분리하고, 모듈 간 직접 참조 대신 서비스 인터페이스를 통해 통신한다.
- 이유: 모듈 간 의존을 서비스 인터페이스의 메서드 계약(파라미터°§반환값)만으로 한정하여 결합도를 최소화한다.

ADR-SYS-002 인증 방식: JWT(Access + Refresh Token) 선택

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	Auth 모듈, 전 API 엔드포인트

배경 (Context)

온워크는 PC 웹 브라우저와 모바일 앱을 동시에 지원해야 하며, 향후 수평 확장(서버 인스턴스 추가)을 고려해야 한다. 인증 토큰 방식으로 세션 기반과 JWT 기반을 비교 검토하였다.

방식	장점	단점
① JWT ★ 채택	Stateless → 서버 수평 확장 용이 · 모바일에서 쿠키 관리 불필요	토큰 탈취 시 만료 전까지 무효화 어려움 → Redis 블랙리스트로 완화
② 세션 기반	서버 측 즉시 무효화 가능 · 구현 단순	서버 수평 확장 시 세션 공유 필요(Sticky Session 또는 중앙 세션 저장소) · 모바일 쿠키 처리 번거로움

결정 (Decision)

JWT 기반 인증을 채택한다. Access Token(유효기간 30분)과 Refresh Token(유효기간 7일)을 이중으로 발급하며, 비밀번호는 bcrypt로 해시 저장한다.

5회 연속 로그인 실패 시 30분 잠금을 적용하고, 로그아웃 또는 강제 만료 시에는 Access Token을 Redis 블랙리스트에 등록하여 즉시 무효화한다.

이유 (Rationale)

- Stateless 구조로 서버 수평 확장 시 세션 공유 인프라(Redis Cluster, Sticky Session) 불필요
- 모바일 클라이언트에서 쿠키 없이 Authorization 헤더로 토큰 전달 가능
- Access Token 단기화(30분)로 탈취 피해 범위를 제한하면서, Refresh Token(7일)으로 사용자 재로그인 불편을 최소화
- 세션 기반 방식은 서버 확장 시 Sticky Session 또는 별도 세션 저장소 필요로 운영 복잡도가 증가함

대가 (Consequences)

감수하는 트레이드오프

- 토큰 탈취 시 Access Token 만료(최대 30분) 전까지 서버 측 즉시 무효화가 기본적으로 불가
- Refresh Token 유효기간(7일) 내 탈취 시 세션 하이재킹 위험 → HTTPS 강제 및 HttpOnly 쿠키 또는 Secure Storage 저장 정책 필요

완화 방안

- 로그아웃·강제 만료 이벤트 발생 시 Access Token을 Redis 블랙리스트에 등록, 남은 유효기간 동안 요청 차단
- Refresh Token은 DB에 저장하여 서버 측 폐기 가능 (단방향 해시 저장)
- RBAC(역할 기반 접근 제어) 5개 레벨과 연동하여 권한 변경 즉시 토큰 무효화 처리

설계 원리 적용 명세

OCP (개방-폐쇄 원칙)

- RBAC 5개 레벨 구조를 인터페이스로 추상화하여, 새 권한 레벨 추가 시 기존 인증 로직 수정 없이 확장 가능하도록 설계한다.

→ 이유: 권한 체계가 확장되어도 JWT 발급, 검증 로직은 변경되지 않는다.

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	인사모듈

배경 (Context)

신규 입사, 퇴사, 정보 변경 등 인사 이벤트는 경영진의 승인을 거쳐야 공식 효력이 발생한다. 승인 이전에 users 테이블에 데이터를 즉시 반영하면, 승인 취소 시 롤백 로직이 복잡해지고 데이터 일관성(Consistency)이 훼손될 위험이 있다. 또한 PENDING 상태의 직원이 시스템에 로그인하거나 급여·근태 집계에 포함되는 문제가 발생할 수 있다.

방식	장점	단점
① hr_change_requests PENDING 보관 후 승인 시 실반영 ★ 채택	상태 관리 단순(PENDING→APPROVED 단방향)·롤백 로직 불필요·감사 추적 가능	조회 API에 PENDING 필터 로직 추가 필요
② 즉시 반영 후 승인 플래그로 구분	구현 단순	승인 취소 시 롤백 처리 복잡·데이터 일관성 훼손 위험
③ 이벤트 소싱 패턴	완전한 이력 추적 가능	구현 복잡도 과다(4인 팀 규모 부적합)

결정 (Decision)

hr_change_requests 테이블에 PENDING 레코드만 생성하고, 승인 완료 시점에만 users / user_credentials 테이블에 데이터를 실반영한다.

이유 (Rationale)

- 경영진 승인 전에는 공식 직원으로 간주되지 않아야 하므로 로그인·급여 집계 차단이 자연스럽게 구현된다.
- PENDING → APPROVED 단방향 전환으로 상태 관리가 단순해진다.
- 반려(REJECTED) 시 hr_change_requests 레코드만 갱신하면 되므로 롤백 로직 불필요.
- employee_change_histories 테이블에 before_value / after_value JSON을 기록하여 감사 추적(Audit Trail)이 가능하다.

대가 (Consequences)

감수하는 트레이드오프

- 승인 처리 지연 시 경영지원팀이 실제 시스템과 임시 데이터를 별도로 관리해야 하는 부담 발생.
- 조회 API에서 PENDING 건을 별도 필터링하는 로직이 추가로 필요..

완화 방안

- PENDING 건 목록을 경영지원팀 대시보드에서 실시간 확인할 수 있도록 전용 뷰를 제공하고, 승인 지연 알림 기능을 운영한다.
- API Gateway 또는 서비스 레이어에 공통 PENDING 필터 미들웨어를 구현하여 중복 로직을 최소화한다.
- 하위 사용자 혼동을 방지한다.

설계 원리 적용 명세

SRP (단일 책임 원칙)

- hr_change_requests 테이블 — PENDING 상태 요청 보관만 담당
 - employee_change_histories 테이블 — before_value / after_value 감사 이력 추적만 담당
 - users / user_credentials 테이블 — 승인 완료 후 실데이터 반영만 담당
- 이유: 세 테이블이 각각 단일 책임을 가지므로, 승인 정책 변경 시 변경의 파급 범위가 hr_change_requests 처리 로직 내부로 한정된다.

OCP (개방-폐쇄 원칙)

- 인사 변경 유형(CREATE, UPDATE, RESIGN 등)을 change_type 필드로 구분하고, 유형별 처리 로직을 독립 핸들러로 분리한다.
- 이유: 새 변경 유형(예: 부서 이동) 추가 시 기존 핸들러 수정 없이 새 핸들러만 추가하면 된다.

ADR-HR-002 인사 변경 요청에 따른 반려(REJECTED) 구조 유지

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	인사 모듈

배경 (Context)

휴가관리 모듈은 취소 권한을 신청자 본인에게만 부여하고 팀장 반려 없이 승인/보류만 구현하였다. 반면 인사 변경은 경영진이 내용 검토 후 반려 사유를 명시해야 요청자가 수정·재신청할 수 있다. 두 모듈이 동일한 승인 패턴을 공유할 경우, 인사 변경의 특수성(법적 기록, 복잡한 정보)을 반영하기 어렵다.

방식	장점	단점
① 인사관리에 반려(REJECTED) 상태 및 반려 사유 필드 별도 유지 ★ 채택	반려 사유 명시로 재신청 가능·employee_change_histories에 감사 추적·모듈 간 정책 독립	인사·휴가 승인 엔드포인트 별도 설계로 코드 중복 가능성
② 반려 없이 승인/보류만 구현	휴가관리와 동일한 흐름으로 개발 공수 절감	반려 사유 기록 불가·수정 재신청 안내 어려움
③ 공통 Approval 서비스로 통합	코드 재사용·단일 승인 API	두 모듈 정책 차이를 파라미터로 처리하는 복잡도 과다

결정 (Decision)

인사관리 모듈은 PENDING → APPROVED / REJECTED 상태 전이를 유지하며, 반려 시 사유 입력을 필수로 한다. 휴가관리와 별도 승인 정책을 적용한다.

이유 (Rationale)

- 신규 입사·퇴사·정보 변경은 법적·급여적 영향이 크므로 경영진이 반려 사유를 명시해야 경영지원팀이 수정 후 재신청이 가능하다.
- 반려 사유를 employee_change_histories에 기록함으로써 감사(Audit) 추적이 가능하다.
- 모듈 간 정책 분리(SoC)를 통해 향후 정책 변경 시 다른 모듈에 영향 없이 독립적으로 수정 가능하다.

대가 (Consequences)

감수하는 트레이드오프

- 인사 변경 API와 휴가 API의 승인 엔드포인트가 별도로 설계되어 코드 중복 가능성 존재.
- 프론트엔드에서 모듈별 승인 버튼 UI를 다르게 구성해야 하는 작업 필요.

완화 방안

- 공통 승인 처리 로직(상태 전이, 히스토리 기록)은 공유 유틸리티 함수로 추출하고, 반려 사유 필수 여부 등 정책 차이는 파라미터로 제어한다.
- 프론트엔드 승인 컴포넌트를 재사용 가능한 공통 컴포넌트로 설계하되, 반려 사유 입력 필드를 옵셔널 슬롯으로 구성하여 모듈별 요구사항을 수용한다.

설계 원리 적용 명세

OCP (개방-폐쇄 원칙)

- 승인 처리 핸들러를 ApprovalHandler 인터페이스로 추상화한다. HrApprovalHandler는 반려 사유 필수 정책을 캡슐화한다.
→ 이유: 향후 다른 모듈에서 별도 승인 정책이 필요할 때 기존 핸들러 수정 없이 새 핸들러만 추가하면 된다.

ADR-HR-003 퇴사 처리 시 계정 삭제 대신 비활성화 (INACTIVE/RESIGNED)

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	인사 모듈

배경 (Context)

직원 퇴사 시 users 레코드를 물리적으로 삭제하면 근태·휴가·급여 이력 데이터의 참조 무결성이 깨진다. 근로기준법상 근로자 관련 서류는 일정 기간 보존 의무가 있으며, 법적 분쟁 대비 이력 조치가 필요할 수 있다. 퇴직자 데이터를 완전히 삭제하면 향후 세금 정산·퇴직금 계산 시 히스토리 재구성이 불가능해진다.

방식	장점	단점
① users.status를 RESIGNED로 변경(소프트 딜리트) ★ 채택	이력 보존·FK 참조 유지·법적 보존 의무 충족·RESIGNED 로그인 자동 차단	레코드 누적으로 조회 성능 저하 가능·목록 API에 RESIGNED 필터 로직 필요
② 퇴사 즉시 users 레코드 물리 삭제	DB 용량 절감·테이블 경량화	이력 손실·FK 무결성 오류 위험·법적 보존 의무 위반 가능
③ 별도 아카이브 테이블로 이동	운영 테이블 경량화·이력 보존 가능	조회 쿼리 복잡도 증가

결정 (Decision)

퇴사 승인 시 users.status를 RESIGNED로 변경하고 resign_date / resign_reason을 기록한다. 물리 삭제는 수행하지 않으며, 퇴직자의 근태·휴가 이력은 원본 테이블에 보존한다.

이유 (Rationale)

- 근태·휴가·보상휴가 이력과의 FK 참조가 유지되어 데이터 일관성을 보장한다.
- RESIGNED 상태 직원은 로그인에 차단되어 시스템 접근 불가 상태가 자동 유지된다.
- 법적 보존 의무를 별도 아카이브 없이 충족하며, 급여·법무팀 요청 시 즉시 조회 가능하다.
- UC-HR-03 흐름에서 퇴사 처리 후 보상휴가 소급 부여(UC-LEAVE-03)와 자연스럽게 연계된다.

대가 (Consequences)

감수하는 트레이드오프

- RESIGNED 직원 레코드가 테이블에 계속 남아 시간이 지남에 따라 조회 성능 저하 가능 → 인덱스(status, resign_date) 추가로 완화.
- 직원 목록 조회 API에서 RESIGNED 필터 제외 로직을 명시적으로 처리해야 함.

완화 방안

- users 테이블에 (status, resign_date) 복합 인덱스를 추가하여 RESIGNED 레코드 조회 성능 저하를 완화한다.
- 직원 목록 조회 공통 Repository/DAO 레이어에 status != 'RESIGNED' 조건을 기본 필터로 적용하여 누락 없이 일관되게 처리한다.

설계 원리 적용 명세

SRP (단일 책임 원칙)

- users 테이블은 status 필드(ACTIVE / RESIGNED)로 재직 여부만 관리한다.
 - 근태, 휴가, 보상휴가 이력은 각 원본 테이블이 독립적으로 보존한다.
- 이유: 퇴사 처리 로직이 users 테이블의 status 변경만으로 완결되므로, 이력 조회 로직은 수정 없이 그대로 유지된다.

고응집 / 저결합

- RESIGNED 필터를 공통 Repository/DAO 레이어에서 기본 조건으로 적용하여 각 서비스가 개별적으로 필터 로직을 중복 보유하지 않도록 한다.
- 이유: 필터 기준 변경 시 Repository 한 곳만 수정하면 되므로 결합도가 최소화된다.

ADR-ATT-001 결근 감지: 자정 배치 자동 처리

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	근태 모듈

배경 (Context)

당일 출근 기록이 없는 직원을 결근으로 처리해야 한다. 결근은 직원이 아무런 액션을 취하지 않은 경우이므로 시스템이 자동으로 감지해야 한다. 실시간 감지와 자정 배치 처리 두 가지 방식을 검토하였다.

방식	장점	단점
① 자정 배치 처리 ★ 채택	구현 단순 · 하루 1회 실행으로 서버 부하 최소 · 결근 확정 시점이 명확(자정)	자정 이전에는 결근 여부를 팀장이 실시간으로 인지 불가
② 소정근로 종료 시각 기준 실시간 감지	팀장이 결근자를 즉시 인지 가능	근무그룹별 종료 시각이 다를 경우 스케줄러 복잡도 급증 · 휴가, 출장 예외 처리 필요 · 오감지 위험

결정 (Decision)

결근 처리는 매일 자정에 배치 잡으로 실행한다. 배치는 당일 `daily_work_records`에 출근기록(`clock_in_at`)이 없는 직원을 조회하여 `status=ANOMALY`, `anomaly_type=ABSENT`로 레코드를 자동 생성한다. 생성된 결근 건은 팀 근태 이상자 목록에 등록되어 팀장이 익일 확인 및 처리할 수 있다.

이유 (Rationale)

- 결근은 "하루가 끝난 시점"에 확정할 수 있는 이벤트로, 자정을 기준으로 하루 1회 처리하는 것이 결근의 의미와 일치함
- 실시간 감지는 근무그룹마다 소정근로 종료 시각이 다를 수 있어 스케줄러 로직이 복잡해지고, 휴가·외근·출장 등의 예외 상태를 실시간으로 필터링하는 부담이 큼
- 온워크 타깃(20인 규모)에서 결근 확인이 당일 실시간으로 이루어져야 할 긴박한 운영 요구가 없으며, 팀장이 익일 이상자 목록에서 확인하는 흐름으로 충분함
- 하루 1회 배치는 서버 부하가 최소화되며, 실행 시점과 처리 대상이 명확하여 장애 발생 시 재실행이 용이함

대가 (Consequences)

감수하는 트레이드오프

- 자정 이전에는 결근 건이 이상자 목록에 등록되지 않아 팀장이 당일 중 결근자를 시스템에서 실시간으로 파악할 수 없음
- 배치 잡 실행 실패 시 해당 일자의 결근 레코드가 누락될 수 있음

완화 방안

- 배치 잡 실행 결과를 로그로 기록하고, 실패 시 관리자에게 알림을 발송하여 수동 재실행이 가능하도록 운영 절차를 마련

설계 원리 적용 명세

SRP (단일 책임 원칙)

- `AbsenceBatchJob` — 자정 결근 감지 및 레코드 생성만 담당
 - `MonthlySummaryJob` — 월말 집계 스냅샷 생성만 담당
 - `NotificationJob` — 보상휴가 소멸 임박 경고 발송만 담당
- 이유: 하나의 배치 잡이 결근 감지 + 집계 + 알림을 모두 수행하면 알림 채널 변경 시 배치 잡 전체를 재배포해야 한다. 각 잡을 독립 분리하면 변경의 파급 범위가 해당 잡 내부로 한정된다.

고응집 / 저결합

- 결근 감지, 집계, 알림을 각각 독립 잡으로 분리하여 잡 간 직접 의존 없이 결과 데이터(`daily_work_records`)를 통해 간접 연결한다.
- 이유: 집계 로직 변경이 결근 감지 잡에 영향을 주지 않는다.

ADR-ATT-002 미승인 시간외 근로: 이상 기록 후 사후 확정 구조

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	근태 모듈

배경 (Context)

소정근로 종료 시각 이후 퇴근한 직원의 시간외 근로를 처리해야 한다. 사전 승인된 `overtime_requests`가 없는 상태에서 초과 퇴근이 발생하는 경우가 빈번한데, 이를 즉시 차단할지, 기록 후 사후에 처리할지를 검토하였다.

방식	장점	단점
① 이상 기록 후 팀장 사후 확정 ★ 채택	실무 상황 유연하게 수용 · 실근로 인정 여부를 팀장이 맥락에 따라 판단 가능 · 근로 사실 자체는 누락 없이 기록	사후 처리 누락 시 이상 건이 방치될 수 있음
② 사전 승인 없으면 시스템 종료	미승인 야근 원천 방지	긴급 업무 등 현실적 예외 상황을 시스템이 처리 불가 · 직원 불만 및 UX 저하
③ 별도 처리 없이 정상 기록	구현 단순	미승인 야근이 감지되지 않아 관리 공백 발생 · 급여·보상휴가 산정 기준 불투명

결정 (Decision)

사전 승인된 `overtime_requests`가 없는 상태에서 소정근로 종료 시각 이후 퇴근이 발생하면, 서버는 `daily_work_records.status`를 `ANOMALY`로 기록하고 `work_anomalies`에 `anomaly_type=UNAPPROVED_OVERTIME` 레코드를 삽입한다. 퇴근은 정상적으로 처리되며 직원에게 "팀장에게 사후 승인을 요청하세요" 안내를 반환한다. 이후 팀장이 해당 건을 검토하여 실근로 인정 여부(`overtime_approved`)를 확정한다.

이유 (Rationale)

- 긴급 장애 대응, 마감 업무 등 사전에 시간외 근로를 신청할 수 없는 현실적 상황이 20인 규모 조직에서도 빈번하게 발생함
- 시스템을 종료시키는 방식은 실제 근로 사실을 시스템에서 누락시켜 근로기준법상 근로시간 기록 의무와 충돌할 수 있음
- 기록 후 사후 확정 구조는 실근로 사실을 보존하면서도 팀장이 맥락(업무 긴급도, 본인 지시 여부)을 파악하여 인정 여부를 결정할 수 있어 실무 적합성이 높음
- 팀장의 확정 시 실근로 인정(`overtime_approved=true`) 또는 미인정(`overtime_minutes=0`) 중 선택하게 함으로써 보상휴가 산정 기준을 명확히 유지

대가 (Consequences)

감수하는 트레이드오프

- 팀장이 사후 확정을 누락하거나 지연할 경우 UNAPPROVED_OVERTIME 이상 건이 장기 미처리 상태로 방치될 수 있음
- 월 마감 시점까지 미처리 건이 남아 있으면 해당 직원의 시간외 근로가 보상 없이 종료될 위험이 있음

완화 방안

- 팀장 대시보드에 미처리 이상자 수 카드를 상단에 상시 노출하여 처리 누락을 방지
- 월 마감(monthly_summaries 생성) 전 미처리 UNAPPROVED_OVERTIME 건이 존재하면 팀장에게 마감 전 처리 알림을 발송하여 누락 없이 확정하도록 강제

설계 원리 적용 명세

OCP (개방-폐쇄 원칙)

- 이상 유형(anomaly_type)을 열거형으로 관리하여 새 이상 유형(예: 지각, 조기퇴근) 추가 시 기존 처리 로직 수정 없이 확장 가능하도록 설계한다.
→ 이유: 각 이상 유형별 처리 핸들러를 독립 구현체로 추가하면 된다.

고응집 / 저결합

- 미처리 이상 건 알림 발송은 NotificationPort(인터페이스)를 통해 AttendanceService와 분리한다.
→ 이유: 알림 채널 변경이 근태 이상 감지 로직에 영향을 주지 않는다

ADR-LVE-001 휴가 승인 흐름: 단일 승인 + 부재 시 대리자 자동 위임

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	휴가 모듈, 알림 모듈

배경 (Context)

온워크는 20인 규모 회사를 대상으로 하며, 팀장이 승인자, 경영지원팀장이 대리자 역할을 담당한다. 휴가 승인 흐름은 팀장 부재 시 처리 지연이 발생하는 Pain Point를 해결해야 하며, 벤치마킹(다우오피스, 네이버웍스, flex)을 통해 세 가지 설계 옵션을 비교 검토하였다.

옵션	장점	단점
① 단일 승인 + 대리자 자동 위임 ★ 채택	팀장 부재 시 자동 위임으로 대기 제로 · 승인 경로 예측 가능 · 20인 조직에 적합한 단순성	부재 판단 기준을 경영진이 시스템에서 설정해야 함 · 다단계 결재(3인 이상 순차 승인)가 필요한 대규모 조직에는 미대응
② 대리자 미지정 단일 승인 (팀장만)	최단 처리 경로 · 구현 단순	팀장 부재 시 승인 불가 · 경영진 인지 불가 · 보상휴가·연차축진 등 정책 통제 약함
③ 자유 결재선 설정 (신청자 지정)	유연성 최대 · 다양한 조직 구조 대응	승인 누락·임의 우회 위험 · 소규모 조직에서 오버엔지니어링 · 감사 추적 복잡

결정 (Decision)

팀장 단독 승인을 채택한다. 승인 행위는 항상 1인이 수행하며, 팀장 부재 시에는 사전 지정된 대리자(경영지원팀장)에게 승인 권한이 자동 위임된다. 대리자도 부재할 경우에는 경영진(대표이사 또는 부대표이사)이 최종 대리자로 고정되어 승인 누락을 방지한다.

팀장 부재 여부는 휴가 신청 시점에 `leave_requests` 및 `daily_work_records`를 실시간 조회하여 자동 판단한다. 팀장·대행자 모두 부재로 판단될 경우, 시스템은 코드 레벨 fallback으로 경영진(CEO 또는 VP)을 최종 승인자로 자동 배정한다. 대리자 처리된 건은 `leave_requests.is_delegated = TRUE`로 기록하여 감사 추적이 가능하도록 설계한다.

이유 (Rationale)

- 1차 발표 CJM에서 식별한 Pain Point: 네이버웍스 벤치마킹 결과, 대리자를 팀장이 수동으로 지정해야 해 휴가 전 직접 설정하지 않으면 대기 상태가 발생함
- 20인 소규모 조직에서는 조직 계층이 단순하므로 단일 승인(+ 자동 위임) 흐름이 실무에 충분하며, 자유 결재선 구성은 오버엔지니어링
- 대리자 미지정 단일 승인(팀장만)은 팀장 부재 시 처리 지연이 불가피하고, 경영진이 전사 휴가 현황을 통제하기 어려움. 온워크는 대리자를 시스템이 자동 전환함으로써 이를 해결
- 유저 스토리 US-204(승인자 자동 전환)의 인수 조건과 직접 연결: "부재 판단 기준은 경영진이 기본설정에서 지정, 대행 건임을 화면에 별도 표시"

대가 (Consequences)

감수하는 트레이드오프

- 다단계 결재(팀장 → 부서장 → 경영진 순차 승인)가 필요한 대규모 조직에는 대응 불가 → 온워크 타킷(20인)에서는 해당 없음
- 부재 판단 기준(설정 N일)을 경영진이 관리해야 하므로, 설정 미비 시 자동 위임이 작동하지 않을 수 있음

완화 방안

- 대리자 처리된 건임을 화면에 명시적으로 표시하여 감사 추적 가능하도록 설계 (is_delegated 플래그)
- 팀장·대리자 동시 부재 시 경영진 대행 경로를 fallback으로 고정하여 승인 누락 방지
- 보상휴가 소멸 임박 경고(US-205)와 연동하여 경영진이 전사 휴가 현황을 선제적으로 파악할 수 있도록 알림 설계

설계 원리 적용 명세

OCP (개방-폐쇄 원칙)

- 승인자 결정 로직을 ApproverResolver 인터페이스로 추상화한다. DefaultApproverResolver(팀장 → 대리자 → 경영진 순)가 기본 구현체가 된다.
→ 이유: 향후 다단계 결재가 필요한 경우 새 Resolver 구현체만 추가하면 기존 LeaveService 코드는 수정하지 않아도 된다.

고응집 / 저결합

- is_delegated 플래그 기록과 알림 발송은 위임 처리 완료 후 트랜잭션 외부에서 비동기로 실행한다.
→ 이유: 알림 발송 실패가 휴가 승인 트랜잭션에 영향을 주지 않는다.

ADR-LVE-002 휴가 승인 처리: 반려 없이 승인/보류만 구현

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	휴가 모듈 (UC-LEAVE-02)

배경 (Context)

온워크는 휴가 신청에 대해 팀장이 승인 또는 보류만 할 수 있으며, 반려(Reject) 기능은 제공하지 않는다. 다우오피스·네이버웍스 등 경쟁 서비스는 팀장이 신청을 반려하면 직원이 재신청해야 하는 구조인 반면, 온워크는 이를 의도적으로 배제하였다. 이 결정의 근거와 트레이드오프를 기록한다.

옵션	장점	단점
① 승인/보류만 구현 ★ 채택	근로기준법 취지 부합 · 직원 재신청 부담 없음 · 보류로 커뮤니케이션 유도	반려에 익숙한 사용자에게 초기 혼선 가능 · 보류 사유 입력 누락 시 실효성 저하
② 승인/반려/보류 모두 구현	기존 그룹웨어 사용자에게 친숙한 UX	직원이 반려 후 재신청해야 함 · 근로기준법 제60조 5항(연차 사용 촉진 의무) 취지와 충돌 가능

결정 (Decision)

휴가 신청에 대한 팀장의 처리 상태를 APPROVED(승인)와 ON_HOLD(보류) 두 가지로만 한정한다. 반려(REJECTED) 상태는 시스템에 구현하지 않는다. 보류 시에는 사유 입력을 필수로 하여 팀장이 직원에게 사유를 명확히 전달하도록 강제한다.

이유 (Rationale)

- 근로기준법 제60조 5항은 사용자가 근로자의 연차 사용을 제한할 수 없음을 명시한다. 반려 기능은 사실상 연차 사용 거부 수단으로 악용될 소지가 있어 법적 취지에 반할 수 있다.
- 보류(ON_HOLD)는 "일정 조율이 필요한 상태"를 명시적으로 표현하며, 팀장이 사유를 필수 입력하게 함으로써 직원과의 커뮤니케이션을 시스템이 강제한다. 이는 반려보다 협력적인 방식으로 일정을 조율하는 온워크의 차별화 포인트다.
- 다우오피스·네이버웍스 벤치마킹 결과, 반려 후 재신청 구조는 직원에게 불필요한 반복 작업을 유발한다는 Pain Point가 확인되었다.

대가 (Consequences)

감수하는 트레이드오프

- 반려에 익숙한 사용자는 "왜 반려가 없지?"라는 초기 혼선을 경험할 수 있다.
- 팀장이 보류 사유를 성의 없이 입력하거나 반복 보류할 경우 사실상 반려와 동일한 효과가 발생할 수 있다.

완화 방안

- 온보딩 화면 및 팀장 대상 가이드에 "온워크는 보류로 일정을 조율한다"는 안내를 명시하여 초기 혼선을 예방한다.
- 보류 사유는 API 레벨에서 max 500자 필수 입력으로 강제하며, 프론트엔드에서도 사유 미입력 시 확인 버튼을 Disabled 처리한다.

설계 원리 적용 명세

SRP (단일 책임 원칙)

- LeaveService — APPROVED / ON_HOLD 상태 전이 로직만 담당
- 보류 사유 유효성 검증(max 500자 필수) — API 레벨 Validator가 단독으로 담당
→ 이유: 보류 사유 정책 변경(글자 수 제한 조정 등) 시 LeaveService 코드를 수정할 필요가 없다.

보상휴가 다중 부여: 배열 입력 + 부분 성공(Partial Success) 허용

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	휴가 모듈 (UC-LEAVE-03, POST /api/v1/leave-grants)

배경 (Context)

경영진이 야근·특별 기여 등의 사유로 보상휴가를 수동 부여할 때, 실무에서는 팀 단위로 한 번에 여러 명에게 동일 조건으로 부여하는 경우가 많다. API 설계 시 단건(1명씩) 호출과 다중(배열) 호출 두 가지 방식을 검토하였다.

옵션	장점	단점
① 배열 입력 + Partial Success ★ 채택	한 번의 API 호출로 다수 부여 완료 · 퇴직자 포함 시 해당 건만 실패하고 나머지 정상 반영 · 경영진 UX 효율 극대화	부분 성공 응답 파싱 로직이 클라이언트에 필요 · 실패 건 재처리 UI 설계 필요
② 단건 호출 (1명씩)	API 구현 단순 · 응답 구조 명확	10명 부여 시 10회 호출 필요 · 네트워크 비용 증가 · 경영진 UX 저하

결정 (Decision)

POST /api/v1/leave-grants의 user_id를 배열로 받아 한 번의 호출로 다수에게 부여한다. 처리 결과는 results[] 배열로 반환하며, 일부 직원 처리 실패(예: 퇴직자 소급 부여 실패) 시 해당 건만 실패 처리하고 나머지는 정상 반영하는 Partial Success 방식을 채택한다.

이유 (Rationale)

- UC-LEAVE-03 기본 흐름에서 경영진은 팀/부서 단위로 체크박스를 선택해 일괄 부여하는 시나리오가 핵심이다. 단건 호출 구조는 이 UX를 구현하려면 클라이언트에서 반복 호출해야 하므로 네트워크 비용과 실패 처리 복잡도가 급증한다.
- 퇴직자가 목록에 포함될 수 있는 현실적 상황(소급 부여 시도)을 고려할 때, 전체 롤백보다 해당 건만 실패 처리하는 Partial Success가 경영진 입장에서 훨씬 실용적이다.

대가 (Consequences)

감수하는 트레이드오프

- Partial Success 응답(results[] + success_count + fail_count)을 클라이언트가 파싱하고 실패 건을 별도로 표시해야 하므로 프론트엔드 구현 복잡도가 단건 호출 대비 높다.
- 부분 성공 상황에서 경영진이 실패 건을 인지하지 못하고 넘어갈 경우 부여 누락이 발생할 수 있다.

완화 방안

- 부여 완료 화면에서 실패 건을 별도 섹션으로 명확히 표시하고, fail_count > 0인 경우 경고 배지를 노출하여 경영진이 반드시 인지하도록 설계한다.
- 실패 건은 재처리 버튼을 제공하여 경영진이 해당 직원만 선택해 재부여할 수 있도록 한다.

설계 원리 적용 명세

SRP (단일 책임 원칙)

- leave_grants 테이블 — 보상휴가 부여 이력 추적만 담당
 - leave_balances 테이블 — 잔여일수 현재 상태 관리만 담당
 - leave_histories 테이블 — 잔여일수 변동 이력 추적만 담당
- 이유: 세 테이블이 각각 단일 책임을 가지므로 부여 정책 변경이 잔여일수 조회 로직에 영향을 주지 않는다.

OCP (개방-폐쇄 원칙)

- Partial Success 응답 구조(results[], success_count, fail_count)를 공통 BatchResult 인터페이스로 추상화한다.
- 이유: 향후 다른 일괄 처리 API(예: 연차 일괄 부여)에서도 동일한 응답 구조를 재사용할 수 있고, 기존 코드 수정 없이 확장된다.

고응집 / 저결합

- 부여 대상 유효성 검증(RESIGNED 여부 등)은 LeaveGrantValidator로 분리하고, LeaveService의 부여 로직과 독립적으로 동작하도록 설계한다.
- 이유: 유효성 검증 조건 변경 시 LeaveService 내부 로직을 수정하지 않아도 된다.

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	결재/알림 모듈, 휴가 모듈, 근태 모듈, 인사 모듈

배경 (Context)

현재 온워크의 설계에서 팀장은 휴가 승인, 근태 이상자 처리, 시간외 근로 승인 등 여러 승인 절차를 각각 다른 화면에서 처리해야 한다. 이는 1차 발표에서 식별한 팀장 한소희의 Pain Point — "팀원의 요청을 놓치지 않고 처리할 수 있는 일괄 관리 환경" — 를 오히려 악화시킨다.

방식	장점	단점
① 통합 결재함 ★ 채택	처리 대기 건을 유형 구분 없이 한 화면에서 일괄 처리 · 미처리 건수 배지로 인지 부담 최소화	approvals 중간 테이블 필요 · 각 모듈 처리 로직 재사용 설계 필요
② 현행 유지 (메뉴별 분산)	구현 단순	팀장이 3개 메뉴를 순회하며 처리해야 함 · 누락 위험
③ 알림 클릭 → 개별 처리	알림 연동 자연스러움	처리 흐름이 알림 존재 여부에 종속 · 일괄 처리 불가

결정 (Decision)

approvals 통합 테이블을 도입하여 휴가 신청 · 근태 이상 · 인사 변경 요청을 단일 결재함 뷰에서 조회하고 처리한다. 처리 시 각 모듈의 기존 서비스 로직을 내부적으로 재사용(위임 호출)하여 중복 구현을 방지한다.

이유 (Rationale)

- 팀장은 결재함 하나만 확인하면 모든 처리 대기 건을 파악할 수 있어 인지 부담 대폭 감소
- 모듈별 처리 로직은 각 서비스(LeaveService, AttendanceService, HrService)에 위임하므로 결재함은 조회/라우팅만 담당 → SRP 준수

대가 (Consequences)

감수하는 트레이드오프

- approvals 테이블이 각 모듈 PK를 ref_id로 느슨하게 참조 (FK 미설정)하므로 데이터 정합성을 서비스 레이어에서 보장해야 함

완화 방안

- 각 모듈 처리 API 호출 전 ref_id 유효성 검증 로직을 ApprovalService에서 공통 처리

설계 원리 적용 명세

SRP (단일 책임 원칙)

- ApprovalService — 결재함 조회 및 라우팅만 담당
- 실제 비즈니스 처리는 LeaveService, AttendanceService, HrService에 위임한다.
→ 이유: ApprovalService가 LeaveRequest 내부 로직까지 알면 휴가 정책 변경 시 결재 코드도 수정해야 한다.
위임 구조를 유지하면 ApprovalService는 얇은 오케스트레이터로만 동작한다.

OCP (개방-폐쇄 원칙)

- 결재 라우터를 ApprovalHandler 인터페이스로 추상화한다.
- LeaveApprovalHandler, AttendanceApprovalHandler, HrApprovalHandler가 구현체가 된다.
→ 이유: 향후 새 결재 유형(예: 일정 참석 확인) 추가 시 ApprovalService 코드 수정 없이 새 핸들러만 추가하면 된다.

ISP (인터페이스 분리 원칙)

- ApprovalQueryPort(조회: 대기 목록, 상세 조회)와 ApprovalCommandPort(처리: 단건, 일괄 승인)를 분리한다.
→ 이유: 처리완료 탭 같은 읽기 전용 화면은 ApprovalQueryPort만 의존하면 되므로 처리 로직 변경에 영향받지 않는다.

저결합

- ApprovalService는 각 모듈의 엔티티를 직접 참조하지 않고 서비스 인터페이스의 메서드 계약(파라미터, 반환값)만으로 통신한다.
→ 이유: 특정 모듈 DB 스키마 변경이 결재 코드에 영향을 주지 않는다.

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	온보딩 모듈, 인사 모듈

배경 (Context)

온보딩 모듈은 초기 검토 단계에서 NEW_HIRE(신입사원), PROMOTION(승진), TRANSFER(부서 이동), RETURN(휴직 복귀) 총 4가지 유형을 고려하였다. 그러나 RETURN은 휴직 기준 설정 및 배치 처리 등 운영 복잡도가 크고, 조직 변화 적응 중심이라는 점에서 모듈의 핵심 목적과 맞지 않는다. 또한 TRANSFER는 실제 기능·권한 변화가 거의 없어 학습 필요성이 낮다.

이에 따라 온보딩 유형을 NEW_HIRE와 MANAGER_TOUR(권한 상승) 2종으로 축소하는 안과 4종을 모두 지원하는 안, 신입만 지원하는 1종 안을 비교 검토하였다.

방식	장점	단점
① 2종 (NEW_HIRE / MANAGER_TOUR) ★ 채택	메시지 명확 · 트리거 조건 단순 · 구현·테스트 범위 최소화 · UC-HR-02와 직접 연계 가능	향후 TRANSFER·RETURN 도입 시 별도 ADR 및 추가 설계 필요
② 4종 모두 지원	포괄적 지원 · 모든 인사 이벤트 대응	RETURN 휴직 기준 판단 모호 · TRANSFER 권한 변화 없어 학습 필요성 낮음 · 구현 복잡도 과다
③ 신입만 지원 (1종)	구현 단순 · 최소 범위	신입 팀장 권한 상승 시 학습 지원 부재 · 핵심 슬로건과 충돌

결정 (Decision)

온보딩 유형을 다음 2종으로 한정한다.

- NEW_HIRE — UC-HR-02 승인 시 `change_type='CREATE'`인 경우 자동 생성
- MANAGER_TOUR — UC-HR-02 승인 시 RBAC 권한 레벨이 상승(EMPLOYEE → MANAGER)하는 경우 자동 생성

TRANSFER, RETURN 유형은 본 버전에 포함하지 않으며, 데이터 구조는 향후 유형 추가가 가능하도록 `tutorial_code`를 확장 가능한 열거형으로 설계한다.

이유 (Rationale)

- "권한이 늘어나면 OnWork가 알려준다"는 제품 메시지가 두 유형(입사·승진)으로 명확히 전달된다.
- 트리거 조건이 UC-HR-02의 `change_type`과 RBAC 레벨 확인만으로 자동 판정 가능하여, 별도 입력·관리 비용이 발생하지 않는다.
- RETURN은 휴직 종료 시점 판단이 인사 정책마다 달라 자동 트리거에 부적합하며, TRANSFER는 권한·화면 변경이 없어 튜토리얼의 효용이 낮다.
- 20인 규모 조직에서 핵심 학습 이벤트는 입사와 권한 상승 두 가지로 충분히 커버된다.

대가 (Consequences)

감수하는 트레이드오프

- 향후 TRANSFER 또는 RETURN 유형을 추가하려면 별도 ADR 및 트리거 조건 정의가 필요하다.
- 부서 이동 직원에 대한 별도 안내 채널이 없어, 이동 시 변경된 화면·동료 정보는 안내되지 않는다.

완화 방안

- tutorial_code를 확장 가능한 열거형으로 설계하여, 신규 유형 추가 시 데이터 마이그레이션 없이 코드 배포만으로 확장 가능하도록 한다.
- 부서 이동 직원에게는 별도 알림 모듈을 통해 변경 사항을 전달하고, 온보딩 모듈은 권한·학습 중심 이벤트에만 집중한다.

설계 원리 적용 명세

SRP (단일 책임 원칙)

- OnboardingService — 튜토리얼 진행 상태 관리만 담당
- HrService — 인사 변경 승인 및 RBAC 권한 변경만 담당
- OnboardingTriggerHandler — UC-HR-02 승인 이벤트를 수신하여 tutorial_code를 판정하는 역할만 담당

→ 이유: 인사 변경 정책이 바뀌어도 OnboardingService는 영향을 받지 않으며, 튜토리얼 흐름이 변경되어도 HrService는 수정되지 않는다.

OCP (개방·폐쇄 원칙)

- tutorial_code를 열거형으로 관리하여, 새 유형(예: TRANSFER, RETURN) 추가 시 기존 트리거 판정 로직 수정 없이 새 핸들러만 추가하면 된다.

→ 이유: 현재 2종에 한정하더라도 확장 가능 구조를 유지함으로써, 정책 변경에 따른 코드 수정 범위를 새 핸들러 내부로 한정한다.

ADR-APP-003 온보딩 시작 트리거: 시스템 자동 방식 채택

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	온보딩 모듈, 인사 모듈

배경 (Context)

신규 입사자 또는 권한 상승자에게 온보딩 튜토리얼을 노출하기 위해서는 `onboarding_tutorial_progress` 레코드 생성 시점을 결정해야 한다. 트리거 방식으로 시스템 자동 생성, 경영지원팀 수동 트리거, 사용자 직접 시작 세 가지를 비교 검토하였다.

수동 트리거 방식은 인사 승인과 온보딩 시작이 분리된 별도 작업이 되므로, 담당자의 추가 클릭이 필요하고 누락 시 신규 입사자에게 튜토리얼이 노출되지 않는 운영 공백이 발생할 수 있다. 또한 담당자 부재(휴가·명절 등) 시 시작 지연이 누적될 위험이 존재한다.

방식	장점	단점
① 시스템 자동 트리거 ★ 채택	담당자 개입 불필요 · 누락 위험 제거 · 사용자 경험 일관성 · 자동 안내 컨셉 부합	UC-HR-02 사후 조건 추가로 트랜잭션 책임 범위 확장
② 경영지원팀 수동 트리거	권한 분리 명확 · 예외 케이스 수동 통제 가능	담당자 추가 작업 발생 · 누락 위험 · 담당자 부재 시 시작 지연
③ 사용자 직접 시작	사용자 자율성 보장 · 시스템 단순	신규 사용자가 튜토리얼 존재 자체를 인지하지 못할 가능성 · 노출 누락

결정 (Decision)

UC-HR-02(인사정보 변경 요청 처리) 승인 완료 시점에 시스템이 자동으로 `onboarding_tutorial_progress` 레코드를 생성한다.

- `change_type='CREATE' → tutorial_code='NEW_HIRE_TOUR'`
- RBAC 권한 레벨 상승(EMPLOYEE → MANAGER) → `tutorial_code='MANAGER_TOUR'`

생성 시점에 `status='NOT_STARTED'`로 초기화되며, 해당 사용자가 다음 로그인 시 자동으로 튜토리얼이 노출된다. 레코드 생성은 인사 승인 트랜잭션과 분리하여 비동기 이벤트로 처리한다.

이유 (Rationale)

- 인사 승인 시점에 온보딩 대상자가 확정되므로, 동일 시점에 레코드를 생성하면 누락 가능성이 제거된다.
- 담당자의 추가 작업 없이 시작되므로 경영지원팀 업무 부담이 증가하지 않으며, 담당자 부재로 인한 지연도 발생하지 않는다.
- "권한이 변경되면 시스템이 자동으로 학습을 안내한다"는 제품 컨셉과 트리거 방식이 일치한다.
- 이미 OnWork 사용 경험이 있는 사용자에게도 노출되는 케이스는 [그만 보기] 옵션으로 사용자 측에서 해결할 수 있어, 자동 트리거의 부작용 범위가 제한적이다.

대가 (Consequences)

감수하는 트레이드오프

- UC-HR-02 승인 트랜잭션의 사후 조건이 추가되어 책임 범위가 확장된다. 단, 비동기 이벤트로 처리하므로 인사 승인 응답 시간에는 영향이 없다.
- 튜토리얼이 불필요한 사용자(경력직 등)에게도 일률적으로 노출된다.

완화 방안

- 레코드 생성을 인사 승인 트랜잭션 외부의 비동기 이벤트 핸들러에서 처리하여, 생성 실패가 인사 승인에 영향을 주지 않도록 격리한다.
- 튜토리얼 화면에 [그만 보기] 옵션을 제공하여 `status='DISMISSED'`로 전환할 수 있도록 하고, `dismissed_at` 기록으로 사용자 의사를 명시적으로 보존한다.
- 수동 부여 화면은 본 버전에서 제공하지 않으며, 운영상 필요가 확인되면 별도 유스케이스로 추가한다.

설계 원리 적용 명세

SRP (단일 책임 원칙)

- HrService — 인사 변경 승인 및 도메인 이벤트(HrApprovedEvent) 발행만 담당
- OnboardingEventHandler — HrApprovedEvent 수신 및 `tutorial_code` 판정만 담당
- OnboardingService — `onboarding_tutorial_progress` 레코드 생성·조회만 담당

→ 이유: 인사 승인 정책이 변경되어도 온보딩 트리거 로직은 영향을 받지 않으며, 트리거 조건이 추가되어도 인사 모듈 코드는 수정되지 않는다.

OCP (개방-폐쇄 원칙)

- 트리거 판정 로직을 TriggerResolver 인터페이스로 추상화한다. 현재 구현체는 ChangeTypeResolver와 RbacUpgradeResolver 두 종류이며, 새로운 트리거 조건이 필요할 경우 새 Resolver를 추가하여 확장한다.

→ 이유: 향후 다른 인사 이벤트(예: 부서장 임명) 기반 트리거가 필요해질 경우, 기존 핸들러 수정 없이 새 Resolver만 등록하면 된다.

고응집 / 저결합

- 인사 모듈과 온보딩 모듈은 도메인 이벤트(HrApprovedEvent)로만 통신하며, 직접 호출하지 않는다.

→ 이유: 인사 트랜잭션 실패가 온보딩 레코드 생성에, 또는 온보딩 핸들러 오류가 인사 승인에 영향을 주지 않도록 결합도를 최소화한다.

상태	승인됨 (Accepted)
결정일	2025년 5월
결정자	팀 전체 합의
관련 모듈	온보딩 모듈

배경 (Context)

튜토리얼 시나리오(각 단계의 말풍선 내용, 가리킬 UI 요소, 다음 단계 전이 조건)를 어디에 저장할 것인가에 대한 결정이 필요하다. 초기 검토안에서는 "진행률 트래커" 개념으로 접근하여 `onboarding_packages`(개인별 패키지), `onboarding_items`(체크 항목, `completion_rule` 구분), `onboarding_templates`(유형별 기본 항목) 세 테이블로 구성하는 구조를 고려하였다.

그러나 요구사항을 재분석한 결과, 본 모듈의 본질은 "체크리스트 + 진행률 계산"이 아니라 "단방향 가이드 투어"로 확인되었다. 사용자는 항목을 자유롭게 체크하는 것이 아니라 Step 1 → Step 2 → ... 순서로 따라가며, 말풍선 내용·가리킬 버튼·전이 조건은 모든 사용자에게 동일하다. 자동 판정 단계도 별도 items 테이블이 아닌 튜토리얼 흐름의 한 단계로 표현하는 것이 자연스럽다.

이에 따라 시나리오 저장 위치로 다음 세 가지 방식을 비교 검토하였다.

방식	장점	단점
① 코드 기반 ★ 채택	데이터 구조 단순화(테이블 1개) · 자동 판정 로직 불필요 · 디자이너·기획자 협업 용이 · 검증된 라이브러리 재사용	관리자가 DB에서 시나리오 직접 수정 불가 · 코드 배포 필요
② DB 기반 (<code>onboarding_items</code>)	관리자 웹 편집 가능 · 시나리오 수정 시 배포 불필요	테이블 복잡도 증가 · 자동 판정 로직 필요 · 디자인 변경마다 DB 수정
③ 하이브리드 (기본은 코드, 커스텀은 DB)	유연성 최대 · 회사별 커스터마이징 가능	구조 복잡도 과다 · 현재 단계에서 커스터마이징 요구 부재

결정 (Decision)

튜토리얼 시나리오는 데이터베이스가 아닌 프론트엔드 코드에 정의한다.

데이터베이스 — 진행 상태만 저장

`onboarding_tutorial_progress` 테이블 1개만 사용한다. 저장 항목은 `user_id`(누가), `tutorial_code`(어떤 튜토리얼을), `current_step`(어디까지 진행했는지), `status`(현재 상태), `last_shown_at` / `dismissed_at` / `completed_at`(시점 정보)로 한정한다.

프론트엔드 코드 — 시나리오 정의

`Driver.js`, `Shepherd.js`, `React Joyride` 등 검증된 가이드 투어 라이브러리를 활용하여 시나리오를 코드로 정의한다. 예: `const NEW_HIRE_TOUR = [{ step: 1, target: "#welcome", message: "환영합니다..." }, ...]`

이유 (Rationale)

- 데이터 구조가 단순해진다. 테이블 3개에서 1개로, 컬럼 23개에서 10개로 축소된다.
- 자동 판정 로직이 불필요해진다. `AUTO_DATA` / `AUTO_GUIDE` / `MANUAL` 구분이 사라지고, 단계 전이 조건은 프론트엔드 코드가 직접 처리한다.
- 디자이너·기획자와의 협업이 용이하다. 말풍선 위치·색상·문구 변경 시 DB 수정 없이 코드 수준에서 처리할 수 있다.
- 배포 속도가 향상된다. 시나리오 수정이 코드 배포로 완결되며, DB 마이그레이션이 필요하지 않다.

대가 (Consequences)

감수하는 트레이드오프

- 관리자가 운영 화면에서 시나리오를 직접 수정할 수 없다. 시나리오 변경에는 코드 배포가 필요하다.
- 회사별 커스터마이징(예: A사 8단계, B사 5단계)이 어렵다. 단, 본 제품은 자체 구축형이므로 단일 시나리오 운영이 전제된다.

완화 방안

- 튜토리얼 시나리오는 변경 빈도가 낮은 영역으로 분류하여, 코드 배포 주기와 충돌하지 않도록 운영한다.
- 향후 관리자 웹 편집 요구가 발생하면 `onboarding_templates` 테이블을 추가하여 점진적으로 확장 가능하도록 데이터 구조를 보존한다.

설계 원리 적용 명세

SRP (단일 책임 원칙)

- `onboarding_tutorial_progress` 테이블 — 사용자별 진행 상태 보관만 담당
- `OnboardingService` — 진행 상태 조회·갱신만 담당
- 프론트엔드 튜토리얼 라이브러리 — 시나리오 정의 및 단계 전이 UI만 담당

→ 이유: 데이터(진행 상태)와 표현(시나리오 UI)을 분리함으로써, 시나리오 변경이 백엔드에 영향을 주지 않고 진행 상태 정책 변경이 프론트엔드에 영향을 주지 않는다.

OCP (개방-폐쇄 원칙)

- `tutorial_code` 값을 식별자로 사용하여, 새 튜토리얼(예: `REPORTING_TOUR`) 추가 시 기존 테이블 스키마 변경 없이 코드만 추가하여 확장한다.

→ 이유: 튜토리얼 종류가 늘어나도 데이터베이스 구조와 진행 상태 관리 로직은 수정되지 않는다.

고응집 / 저결합

- 진행 상태 저장 책임은 백엔드(테이블 + Service)에, 시나리오 정의 책임은 프론트엔드(라이브러리 + 코드)에 응집시킨다. 양측은 `tutorial_code`와 `current_step`이라는 최소 인터페이스로만 통신한다.

→ 이유: 시나리오 변경이 백엔드에 전파되지 않고, 진행 상태 스키마 변경이 프론트엔드 라이브러리에 영향을 주지 않는다.